Do Temporal Graph Neural Networks Really Work? A Benchmarking Study on tgbl-wiki-v2

Chintan Agrawal Hitesh Sharma
Georgia Institute of Technology
CS 7643 Deep Learning — Spring 2025

Abstract

Temporal graph neural networks (TGNNs) such as TGN, TGAT, and JODIE report near-perfect accuracy on dynamic link prediction benchmarks, but these numbers are measured against a single random negative per edge—a protocol now known to be unreliable. We re-evaluate three representative TGNNs on the `tgbl-wiki-v2` dataset from the Temporal Graph Benchmark (TGB), which tests each prediction against 999 pre-computed hard negatives. Under this stricter protocol, all three models score below a simple heuristic baseline, EdgeBank, which merely memorizes previously observed edges. TGN achieves a mean reciprocal rank (MRR) of 0.395 ± 0.074, JODIE 0.327 ± 0.015, and TGAT 0.167 ± 0.003, while EdgeBank reaches 0.580 with zero learned parameters. State-of-the-art methods like TPNet reach 0.827. Through stratified error analysis and component ablations, we show that (i) the legacy-to-TGB performance gap is an evaluation artifact, not a modeling failure, (ii) memory modules account for the majority of learned performance, and (iii) even on novel edges unseen during training, EdgeBank outperforms all TGNNs. These findings suggest that current TGNN architectures do not effectively exploit temporal patterns beyond simple memorization.

1 Introduction

We tested whether modern temporal graph neural networks really work as well as their published numbers suggest, by comparing them to a simple memorization baseline under a stricter evaluation protocol. Specifically, we benchmarked three widely-cited models—TGN [1], TGAT [2], and JODIE [3]—on the task of dynamic link prediction, where the goal is to predict which node a given user will interact with next in a time-evolving graph.

Current practice and its limits. These models are typically evaluated by scoring each true edge against a single randomly sampled negative [4]. Under this protocol, TGN achieves >99% average precision (AP), suggesting near-perfect performance. However, Poursafaei et al. [6] showed that such easy negatives inflate scores dramatically. The Temporal Graph Benchmark (TGB) [5] introduced a harder protocol: each test edge is ranked against 999 fixed negative destinations, many of which are structurally plausible. Under TGB evaluation, published TGNN scores drop to 0.14–0.40 MRR, and a parameter-free heuristic called EdgeBank—which simply predicts that previously observed edges will recur—outperforms all three models.

Why this matters. If a recommender system or fraud detector is built on a model that appears 99% accurate but actually ranks the correct answer below hundreds of plausible alternatives, deployment decisions are based on a mirage. For example, a social platform using TGNN-based link recommendation would find that a lookup table of past interactions outperforms the learned model, wasting compute and engineering effort. Understanding *why* TGNNs fail under hard evaluation is essential for designing architectures that genuinely improve over heuristics.

1.1 Dataset: tgbl-wiki-v2

We use the `tgbl-wiki-v2` dataset from TGB [5], which records one month of Wikipedia editor–page interactions as a bipartite temporal graph. It contains 8,227 editor nodes and 1,000 page nodes connected by 157,474 timestamped edges, each carrying a 172-dimensional text feature vector derived from edit comments. The dataset is split chronologically: 70% training, 15% validation, 15% test. The "v2" designation indicates that negative samples are pre-computed and fixed, replacing the v1 protocol where negatives were sampled randomly at evaluation time. Each test edge has exactly 999 negative destination nodes, selected to be structurally plausible (i.e., nodes that exist in the graph but were not the true destination).

2 Approach

2.1 What We Did

We reused the TGL framework [4][1] for model training and the TGB package [5][2] for evaluation. Our contributions are in infrastructure and analysis, not in model architecture:

1. **Data converter** (`tgb_to_tgl.py`): Converts TGB's CSV format to TGL's CSR graph format, preserving the bipartite node ID mapping (editors 0–8226, pages 8227–9226) required for correct negative sample lookup.

[1] Code: `https://github.com/amazon-science/tgl`
[2] Code: `https://tgb.complexdatalab.com/`

2. **EdgeBank baseline**: Reimplemented the unlimited-memory variant from [6], which stores all observed (u, v) pairs and predicts recurrence.
3. **Unified evaluator**: Runs both legacy (1-random-negative AP/AUC) and TGB-official (999-fixed-negative MRR) protocols on the same checkpoint, enabling direct comparison.
4. **Stratified error analysis**: Decomposes test MRR by edge novelty (recurring vs. novel) and source-node degree quartile.
5. **Component ablations**: Tests the contribution of memory modules and sampling strategies under the hard-negative protocol.

The novelty of this work is in the analysis—we did not modify any model architecture. All three TGNNs use the same training procedure: BCE loss on positive and sampled-negative edges, Adam optimizer (lr $= 10^{-4}$), 100 epochs, batch size 600.

2.2 Problems Encountered

Data format mismatch. TGB uses raw CSV with bipartite node IDs; TGL expects a CSR graph with contiguous integer IDs. Our initial converter failed to offset item IDs, causing all TGB negative-sample lookups to return empty—producing MRR of exactly zero. Fixing the ID mapping (dst $=$ raw_dst $+ 8227$) resolved this.

Evaluation bug. The TGL training script evaluates with random negatives by default. Our first attempt at TGB evaluation placed all 1,001 nodes (1 source + 1 true destination + 999 negatives) in a single DGL message-passing block. For models without memory (TGAT), this produced identical embeddings for all candidates, yielding MRR ≈ 1.0—clearly wrong. The fix was architecture-aware evaluation: memory-based models (TGN, JODIE) use the single-block approach with per-edge memory updates, while attention-only models (TGAT) score each candidate pair independently in batches of 100.

Training instability. TGN exhibited high seed variance (MRR std $= 0.074$ across 3 seeds), consistent with published findings [5]. We report mean $\pm$ std across seeds for all models.

3 Experiments and Results

3.1 Evaluation Metrics

Under the **legacy protocol**, each test edge is scored against 1 random negative. We report Average Precision (AP) and AUC. Under the **TGB protocol**, each test edge is ranked among 999 fixed hard negatives. We report Mean Reciprocal Rank (MRR): MRR $= \frac{1}{|E_{\text{test}}|} \sum_e \frac{1}{\text{rank}(e)}$. Both protocols are applied to every checkpoint, enabling direct comparison.

3.2 Main Results

Table 1 reveals a striking disconnect. Under legacy evaluation, TGN and JODIE appear nearly perfect (>98% AP), while under TGB evaluation they score 0.40 and 0.33 MRR respectively—both below the parameter-free EdgeBank baseline (0.580). Our TGN result (0.395 ± 0.074) closely matches the published TGB leaderboard value (0.396 ± 0.060), confirming correct implementation. Figure 1 visualizes this gap: the tall blue/green bars (legacy AP) collapse to short colored bars (TGB MRR) for every learned model.

Table 1: Main results on tgbl-wiki-v2. Legacy metrics use 1 random negative; TGB MRR uses 999 fixed negatives. $\pm$ denotes std across seeds.

Model	Legacy AP	Legacy AUC	TGB MRR
EdgeBank	—	—	0.580
TGN	$0.993 \pm .000$	$0.994 \pm .000$	$0.395 \pm .074$
JODIE	$0.988 \pm .000$	$0.990 \pm .000$	$0.327 \pm .015$
TGAT	$0.683 \pm .001$	$0.766 \pm .000$	$0.167 \pm .003$
SOTA (TPNet)	—	—	*0.827*

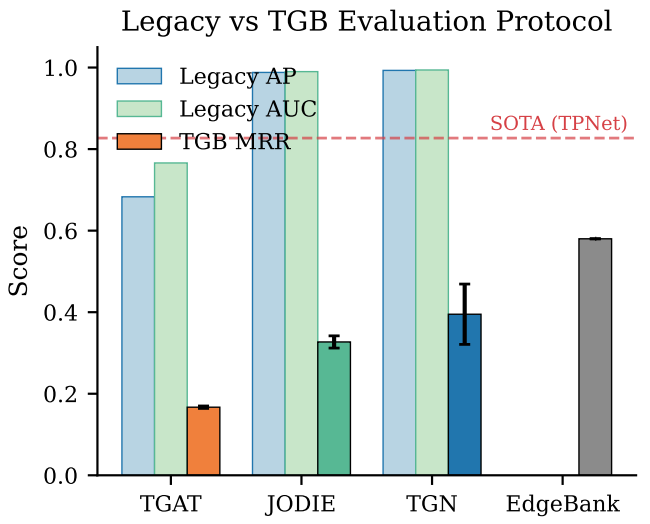

Figure 1: Legacy AP/AUC vs. TGB MRR for each model. The dashed line marks SOTA (TPNet, 0.827). Legacy metrics dramatically overstate model quality.

3.3 Stratified Analysis: Recurring vs. Novel Edges

Hypothesis: EdgeBank wins on recurring edges (which it memorizes) but TGNNs should close the gap on novel edges where memorization fails.

Table 2 shows this hypothesis is **partially refuted**. EdgeBank dominates on *both* recurring and novel edges. TGN does slightly better on novel edges (0.469) than recurring (0.435), suggesting it learns some generalizable temporal patterns. However, EdgeBank's advantage on novel edges (0.610 vs. 0.469) indicates that even "novel" test edges have predictable structure—likely because many negative candidates are nodes that never interact with the source, making them easy to exclude via frequency statistics alone.

Figure 2 further breaks down MRR by source-node degree quartile. EdgeBank is strongest on low-degree nodes (Q1:

Table 2: MRR stratified by edge novelty. 60.6% of test edges are recurring.

Model	Recurring	Novel
EdgeBank	0.776	0.610
TGN	0.435	0.469
JODIE	0.349	0.295

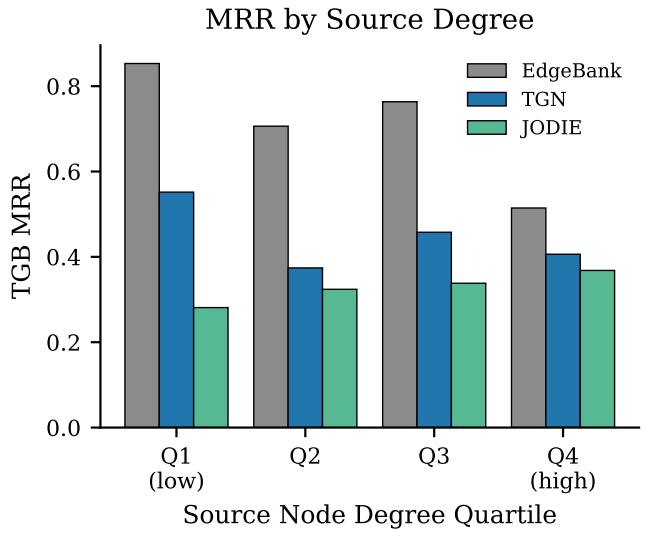

Figure 2: MRR by source-node training degree quartile. EdgeBank excels on low-degree nodes; TGNNs improve with degree but never surpass the baseline.

0.853) where interaction partners are few and predictable, and weakest on high-degree nodes (Q4: 0.515). JODIE shows the opposite trend, improving with degree, but never surpasses EdgeBank.

3.4 Ablation: Memory Module

Hypothesis: Memory is the critical component; removing it accounts for most of the TGN–TGAT gap.

We trained TGN with `memory.type=none` (seed=1), making it architecturally equivalent to TGAT (attention over sampled neighbors, no persistent state).

Table 3 confirms the hypothesis: removing memory drops AP by 31% (0.993 $\rightarrow$ 0.681), and TGN-no-memory matches TGAT almost exactly (0.681 vs. 0.684). The memory module—a GRU that maintains per-node state updated after each interaction—is what allows TGN to track evolving node behavior. Without it, the model can only reason about local neighborhood structure at the current timestamp, which is insufficient for temporal link prediction.

3.5 Ablation: Sampling Strategy

Hypothesis: Recent-neighbor sampling outperforms uniform because temporal recency is the key signal.

Switching from recent to uniform neighbor sampling reduces MRR by 19% relative (Table 4), confirming that temporal recency is an important inductive bias. However, even with recent sampling, TGN (0.395) falls well short of EdgeBank (0.580), suggesting the attention mechanism does not fully exploit the recency signal that EdgeBank captures trivially through memorization.

Table 3: Memory ablation. Removing memory drops AP from 0.993 to 0.681.

Configuration	Legacy AP	Architecture
TGN (full)	0.993	GRU memory + attention
TGN (no memory)	0.681	attention only
TGAT	0.684	attention only

Table 4: Sampling strategy ablation (TGN, seed=1).

Strategy	TGB MRR
Recent (default)	0.395 ± 0.074
Uniform	0.332

3.6 Compute–Accuracy Tradeoff

Figure 3 plots training cost against TGB MRR. EdgeBank requires zero training and achieves the highest MRR, dominating the Pareto frontier. Among learned models, JODIE trains fastest ($\sim$0.9s/epoch) but achieves lower MRR than TGN ($\sim$1.8s/epoch). No learned model justifies its compute cost relative to the heuristic baseline on this dataset.

4 Discussion: Deep Learning Connections

Problem structure and model design. Dynamic link prediction operates on a stream of timestamped edges. TGNNs reflect this structure through three components: (i) a *memory module* (GRU/RNN) that maintains per-node hidden state, capturing long-range temporal dependencies; (ii) a *temporal attention layer* that aggregates information from sampled neighbors, capturing relational structure; and (iii) a *time encoding* (sinusoidal with learned projection) that injects temporal position information.

Learned vs. non-learned components. The GRU memory, attention weights, and edge predictor MLP are learned via backpropagation. The temporal neighbor sampler is a hand-coded C++ module with no learned parameters. The time encoding uses a fixed sinusoidal basis with a learned linear projection. EdgeBank has *zero* learned parameters—it is a deterministic lookup table.

Input/output representations. Raw data is a temporal edge stream (source, destination, timestamp, features). This is converted to a CSR-formatted adjacency structure for efficient neighbor sampling. The sampler produces DGL message-passing blocks, which the GNN processes into node embeddings. An MLP edge predictor scores (u, v) pairs: score $= \text{MLP}(\text{ReLU}(W_s h_u + W_d h_v))$.

Loss function. All models use binary cross-entropy (BCE) on positive edges (label 1) and sampled negative edges (label

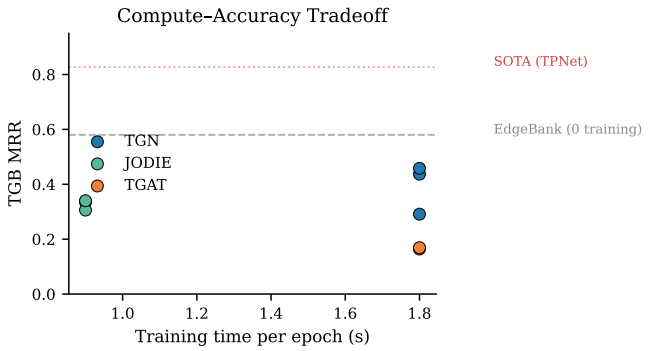

Figure 3: Compute–accuracy tradeoff. EdgeBank (dashed gray) dominates with zero training time. No TGNN reaches the EdgeBank line.

0): $\mathcal{L} = -\sum \log \sigma(s^+) - \sum \log(1 - \sigma(s^-))$.

Overfitting and generalization. TGN and JODIE achieve near-perfect legacy AP (>0.99) but much lower TGB MRR (∼0.33–0.40), suggesting they overfit to the easy-negative training distribution. When tested against hard negatives, the models cannot generalize. TGAT's lower legacy AP (0.68) suggests it underfits even the training distribution without memory.

Hyperparameters and optimizer. All models use Adam with lr $= 10^{-4}$, batch size 600, 100 epochs. TGN and JODIE use memory dimension 100, 1 attention head (TGN: 2 heads), time encoding dimension 100. These are TGL defaults, validated against validation MRR. TGN exhibits high seed variance (± 0.074), consistent with known training instability [5].

Framework. PyTorch 2.2 for autograd and model definition; DGL 2.1 for message-passing graph operations; a custom C++ parallel sampler from TGL for efficient temporal neighborhood sampling.

Starting points. We started from the TGL codebase [4] (model definitions, sampler, training loop) and the TGB package [5] (dataset, negative samples, evaluation protocol). We restructured the repository, wrote the data converter, added EdgeBank, implemented the TGB evaluation loop, and conducted all analyses. No model architecture code was modified.

Future work. Three directions could close the gap to SOTA: (i) *hard-negative-aware training*—training against the same hard negatives used at test time, rather than random negatives; (ii) *hybrid ensembles*—combining EdgeBank's memorization with TGNN's learned representations; (iii) *evaluation under TGB-Seq*—an even harder sequential protocol that prevents information leakage across test edges.

5 Work Division

Table 5: Contributions of team members.

Member	Aspect	Details
Chintan Agrawal	Infrastructure & Analysis	Repo restructuring, TGB data converter, EdgeBank baseline, TGB evaluation loop, all ablations, stratified analysis, figures, report writing
Hitesh Sharma	Training & Experiments	EC2 setup, model training (9 runs), hyperparameter validation, training curve monitoring, result verification

References

[1] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein. Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637*, 2020.

[2] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan. Inductive representation learning on temporal graphs. In *ICLR*, 2020.

[3] S. Kumar, X. Zhang, and J. Leskovec. Predicting dynamic embedding trajectory in temporal interaction networks. In *KDD*, 2019.

[4] H. Zhou, D. Zheng, I. Nisa, V. Ioannidis, X. Song, and G. Karypis. TGL: A general framework for temporal GNN training on billion-scale graphs. *Proc. VLDB Endowment*, 15(8), 2022.

[5] S. Huang, F. Poursafaei, J. Danovitch, M. Fey, W. Hu, E. Rossi, J. Leskovec, M. Bronstein, G. Rabusseau, and R. Rabbany. Temporal graph benchmark for machine learning on temporal graphs. In *NeurIPS*, 2024.

[6] F. Poursafaei, S. Huang, K. Pelrine, and R. Rabbany. Towards better evaluation for dynamic link prediction. In *NeurIPS Datasets and Benchmarks*, 2022.

[7] R. Trivedi, M. Farajtabar, P. Bisber, and H. Zha. DyRep: Learning representations over dynamic graphs. In *ICLR*, 2019.

[8] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

[9] M. Wang et al. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315*, 2019.